

Addendum 1 — Confirmed Hardware Figures and Revised Estimates

Prepared 2026-08-01 · Supersedes: Hardware Assessment §3 and §4; Workflow Architecture §2.2, §2.3 and §4

1. What was confirmed

Question	Answer
Memory configuration	2 × 16 GB DDR5-5600 SODIMM, one module per controller (Controller0 and Controller1)
Dual-channel populated	Yes. This is the optimal arrangement; a single-module configuration would have halved bandwidth
Memory upgradeable	Yes — SODIMM, not soldered. Both slots occupied. See Section 3 for why this does not help
Installed Qwen variant	qwen35moe architecture, 36.0B parameters, Q4_K_M, 262,144 context, Apache 2.0 — confirmed as the MoE build
Unexpected finding	The installed Qwen model reports a vision capability. See Section 4

2. Revised throughput figures

DDR5-5600 across two channels yields 89.6 GB/s theoretical bandwidth. This sits at the bottom of the range assumed in the original assessment, which allowed for the possibility of faster soldered LPDDR5x. All generation-speed estimates must therefore be revised downward by roughly 20 percent.

The figures below assume 55 to 65 percent bandwidth efficiency, which is the realistic band for CPU inference on a mobile platform.

Model / quant	Size	Revised speed	Assessment
GaMS3-12B Q5_K_M	8.45 GB	~6–7 tok/s	Now recommended. The extra throughput matters more than it did under the earlier estimate
GaMS3-12B Q6_K	9.66 GB	~5–6 tok/s	Still defensible. Compare against Q5_K_M on real material before deciding
GaMS3-12B Q4_K_M	7.30 GB	~7–8 tok/s	Fallback only if the corpus proves larger than expected
GaMS3-12B Q8_0	~13 GB	~4 tok/s	No longer recommended. The quality gain over Q6_K will not repay a 30 percent throughput loss
qwen3.6 35B-A3B	24 GB	~15–25 tok/s	Unchanged in role. Fast because only ~3B parameters are read per token

Revision. The original recommendation of Q6_K, with Q8_0 as an upgrade path, is withdrawn. Test Q5_K_M and Q6_K side by side on a representative legal passage and let the certifying translator choose. At these speeds the difference between them is roughly a day of wall-clock time across the corpus.

3. Do not upgrade the memory

The SODIMM finding invites an obvious conclusion that would be wrong. Replacing the two 16 GB modules with 32 GB modules would raise capacity to 64 GB, but it would not raise bandwidth by a single byte per second. Bandwidth is fixed by the transfer rate and the channel count, both of which are already at their practical maximum on this platform.

Since this workload is bandwidth-bound rather than capacity-bound, added capacity buys nothing. Worse, it would only enable running larger models, and a larger model at unchanged bandwidth runs proportionally slower. The 32 GB already installed comfortably holds a 12B model with substantial room for context.

The one configuration worth confirming is already correct: both memory controllers are populated. That was the single change that could have mattered, and it is already in place.

Estimated saving from this finding: the cost of a 64 GB DDR5 SODIMM kit, for no measurable benefit to this project.

4. The vision capability changes the OCR design

The installed Qwen model reports vision among its capabilities. This was not anticipated and it bears directly on the risk identified as dominant in the workflow architecture: silent OCR corruption propagating into fluent, confident, wrong translations.

4.1 Proposal: dual-engine OCR with automated disagreement flagging

A vision-language model reads a scanned page by a completely different mechanism than a traditional OCR engine. Tesseract segments and classifies glyphs; a vision model interprets the image semantically. Their failure modes are largely uncorrelated, and that is the useful property.

Run both over every page, then compare. Where they agree, confidence is high. Where they disagree, flag the page for human inspection. This converts an unbounded proofreading task — checking every page against its image — into a bounded one focused on the passages where something actually went wrong.

Engine	Strength	Characteristic failure
Tesseract (via OCRmyPDF)	Deterministic, fast, reproducible	Mangles diacritics and degraded glyphs; produces nonsense but does not invent
Vision model	Layout-aware; handles stamps, tables, poor scans	Hallucinates plausible text; may silently normalize or omit
Agreement between them	High confidence in the text layer	Correlated failure only on genuinely illegible source

Restrict the automated comparison to what matters most. A character-level diff across whole pages will produce too much noise to act on. Compare instead the extracted numbers, dates, and capitalized tokens — the high-consequence, low-redundancy elements where context will not reveal an error and where disagreement is almost always meaningful.

Verify before designing around this. Vision input through Ollama for GGUF models has historically been inconsistent, and the capability flag does not guarantee working image input on this build. Test it on one scanned page before this becomes load-bearing. If it fails, Surya remains the fallback second engine and the architecture is unchanged.

4.2 A second use: stamps, seals, and signatures

The original architecture proposed adding Gemma 3 12B for describing stamps and signatures. If the installed model's vision path works, that addition becomes unnecessary and the model inventory stays smaller — which is worth something in itself for a workflow that must remain auditable.

Certified translations conventionally describe rather than translate these elements, using bracketed annotations. The vision model can draft those annotations from the page image for the translator to confirm.

5. Context window — cap it deliberately

The installed model advertises a 262,144-token context. This should not be used at anything approaching its full extent. The key-value cache grows with context length and competes directly with the 24 GB the model weights already occupy, against 27 GiB available.

Set an explicit ceiling rather than relying on the default. For segment-level translation with glossary injection, 8,192 to 16,384 tokens is generous; for whole-document context, 32,768 is a sensible upper bound to test against memory pressure.

```
# in an Ollama Modelfile, or per-session via /set parameter
PARAMETER num_ctx 16384

# watch resident size while a job runs
watch -n2 'ollama ps; free -h | head -2'
```

6. Prefill cost — the estimate everyone forgets

Generation speed is only half the throughput picture, and for this workload the other half may dominate. Every segment sent for translation carries a system prompt and an injected glossary. If a 500-token preamble accompanies each of several thousand segments, prefill alone accounts for millions of tokens.

Prefill is compute-bound rather than bandwidth-bound, so it runs faster per token than generation, and it is the one stage where integrated-GPU offload delivers a real gain. Two measures reduce it substantially:

- Keep the system prompt and glossary block byte-identical across requests so the inference engine can reuse the cached prefix instead of recomputing it. Any per-segment variation in the preamble defeats this entirely.
- Batch several segments into one request rather than sending each individually. This amortizes the preamble and generally improves translation consistency, since the model sees neighboring context.

Measure both rates on real material before extrapolating to the corpus. The `--verbose` flag reports prompt eval rate and eval rate separately, and the ratio between them will determine how the batch job should be structured.

7. Corpus planning arithmetic

With generation at roughly 6 tokens per second, planning figures for the scanned-document portion are as follows. These assume translation output only and exclude OCR, prefill, and review time.

Scanned pages	Approx. output tokens	Generation time	Practical schedule
250	~175,000	~8 hours	One overnight run
500	~350,000	~16 hours	Two overnight runs
1,000	~700,000	~32 hours	Three to four overnight runs

Assumes roughly 500 words per page and 1.4 tokens per word. Slovene output may run higher, as the tokenizer is less efficient on morphologically rich text than on English.

The spreadsheet, if the unique-string ratio falls where such data usually falls, should complete in a few hours rather than days. That measurement remains the next concrete step.

The conclusion is unchanged and worth restating: this is comfortably a batch workload for a single machine over one to two weeks of unattended overnight runs. Throughput is not the constraint on this project. Review capacity is.

8. Immediate commands

```
# 1 – pull both candidate quantizations for side-by-side comparison
ollama pull hf.co/mradermacher/GaMS3-12B-Instruct-GGUF:Q5_K_M
ollama pull hf.co/mradermacher/GaMS3-12B-Instruct-GGUF:Q6_K

# 2 – confirm the vision path works before designing around it
ollama run qwen3.6 "Describe the layout of this page, transcribing any
stamp or seal text verbatim." ./scan_sample.png

# 3 – baseline both rates on a real legal passage
ollama run hf.co/mradermacher/GaMS3-12B-Instruct-GGUF:Q5_K_M --verbose

# 4 – measure spreadsheet redundancy (script in Workflow Architecture §7)
```

Item 2 is the highest-value experiment of the four. If vision input works, the OCR verification burden — the largest single risk in this project — becomes substantially more tractable.